

Probabilistic Graphical Models 2 - Inference

A conditional probability query contains

- evidence $E=e$,
- a query $Y \subseteq \text{variables}$,
- a task, the computation of $P(Y|E=e)$.

The following problems are NP-hard: word case

- Given P_{Φ} , X , $x \in \text{Val}(X)$, decide if $P_{\Phi}(X=x) > 0$
- Given $0 < \epsilon < 0.5$, P_{Φ} , X , $x \in \text{Val}(X)$, $e \in \text{Val}(E)$, find $p \in [0,1]$ s.t.
 $|P_{\Phi}(X=x|E=e) - p| < \epsilon$

In the general case, these are easier to solve.

A Maximum a Posteriori (MAP) consists of

- evidence $E=e$,
- the query $Y = \text{all variables} \setminus E$,
- the task to compute $\text{MAP}(Y|E=e) = \underset{\gamma}{\text{argmax}} P(Y=\gamma|E=e)$. not nec. unique

The following problems are NP-hard:

- Given P_{Φ} , find x maximising $P_{\Phi}(x)$,
- Given P_{Φ} , $p \in [0,1]$, decide if $\exists x$ s.t. $P_{\Phi}(x) > p$.

Cluster graphs

A **cluster graph** is an undirected graph with nodes subsets C_i of $\{X_1, \dots, X_n\}$ and an edge between C_i, C_j associated to a fixed **sepsct** $S_{ij} \subset C_i \cap C_j$. Given a set of factors Φ , we associate each ϕ_k to a cluster $C_{\alpha(k)}$ s.t. $\text{Scope}(\phi_k) \subset C_{\alpha(k)}$, and define
$$\psi_i(C_i) = \prod_{k: \alpha(k)=i} \phi_k.$$

A cluster C_i can **pass a message** to a neighbouring cluster C_j by sending
$$\delta_{i \rightarrow j}(S_{ij}) = \sum_{C_i \setminus S_{ij}} \psi_i(C_i) \prod_{k \in \text{Nb}_{i \rightarrow j}} \delta_{k \rightarrow i}(S_{ki}),$$
 where C_j updates $\psi_j(C_j)$ by multiplying by $\delta_{i \rightarrow j}$.
The result is a belief $\beta_j(C_j) = \psi_j(C_j) \prod_{k \rightarrow j} \delta_{k \rightarrow j}$.

This algorithm is approximate. If it converges, it may not converge to the true posterior distribution.

Family preservation: $\forall \phi_k \exists C_i$ s.t. $\text{Scope}(\phi_k) \subset C_i$

to allow information sharing (about X)
to avoid feedback loops

Running intersection: $\forall i, j, X \in C_i \cap C_j \exists$ path C_i to C_j s.t. all clusters and sepscts on the path contain X . In other words, $(C_i \cap X, S_{jk} \cap X)$ is a tree.

The **Bethe cluster graph** has clusters $C_i = \text{Scope}(\phi_i)$, and singletons $\{X_j\}$ for each variable X_j . C_k is connected to X_i iff $X_i \in C_k$. It has both of the above properties, but belief propagation does not work.

A cluster graph is **calibrated** if
$$\sum_{C_i \setminus S_{ij}} \beta_i(C_i) = \sum_{C_j \setminus S_{ij}} \beta_j(C_j).$$

Note that the limit distribution of converged belief propagation is calibrated.

Let $\mu_{ij}(S_{ij}) = \delta_{j \rightarrow i} \delta_{i \rightarrow j}$. We have
$$\frac{\prod_i \beta_i}{\prod_{ij} \mu_{ij}} = \frac{\prod_i \psi_i \prod_{j \rightarrow i} \delta_{j \rightarrow i}}{\prod_{ij} \delta_{i \rightarrow j}} = \prod_i \psi_i = \tilde{P}_{\Phi}.$$

Hence, cluster graph beliefs are an alternative **calibrated** parametrisation of the original (unnormalised) density.
no information loss

A **clique tree** is a cluster graph that is a tree s.t. $S_{ij} = C_i \cap C_j$.

Prop: If a clique tree satisfies the running intersection property then belief propagation converges to the true pdf. When starting at the leaves, it converges after $2(K-1)$ steps, with K the number of cliques.

Theorem: Let T be a clique tree. For $(i, j) \in E_T$, let $W_{\leftarrow ij} = C_i \setminus S_{ij}$. Then T satisfies the running intersection property iff
$$P_{\Phi}(W_{\leftarrow ij} \perp W_{\leftarrow ji} \mid S_{ij}) \quad \forall (i, j) \in E_T.$$

Message passing for MAP ← maximum a posteriori

Recall that for MAP, we want to find

$$\begin{aligned}\arg\max_x P_{\Phi} &= \arg\max \tilde{P}_{\Phi} = \arg\max \prod \phi_k(D_k) \\ &= \arg\max \underbrace{\sum_{\Theta(x_1, \dots, x_n)} \theta_k(D_k)}_{\log \phi_k}\end{aligned}$$

We can use the same algorithms as before, but with max-marginalisation (max-sum) instead of the usual marginalisation of integrating out marginalised variables. We get $\max \sum \theta_k(D_k) = \log(\max \prod \phi_k(D_k)) = \log \max \tilde{P}_{\Phi}$.

If the MAP assignment is unique for each θ_k , this yields the MAP. Otherwise, we can try to perturb parameters to make it unique. There are other algorithms.

Sampling methods

Let $\mathcal{D} = \{x[1], \dots, x[M]\}$ be sampled independently from P . Then

$$E_P(f) \approx \frac{1}{M} \sum f(x[M]).$$

The RHS is called the **empirical expectation**.

Theorem (Hoeffding bound): Let $T_{\mathcal{D}} = \frac{1}{M} \sum x[M]$. Then

$$P_{\mathcal{D}}(T_{\mathcal{D}} \notin [p - \epsilon, p + \epsilon]) \leq 2e^{-2M\epsilon^2}.$$

↑ probability over data sets

Theorem (Chernoff bound): Let $T_{\mathcal{D}} = \frac{1}{M} \sum x[M]$. Then

$$P_{\mathcal{D}}(T_{\mathcal{D}} \notin [(1-\epsilon)p, (1+\epsilon)p]) \leq 2e^{-\frac{1}{3}Mp\epsilon^2}.$$

We can sample a Bayesian network by starting at vertices without parents and then continue to the children (the **topological order**). We call this **forward sampling**.

If we have evidence e , we can use forward sampling but reject any sample when $E \neq e$. This is called **rejection sampling**. The number of samples needed grows exponentially with the number of observed variables.

Note that forward sampling is not applicable to Markov networks in general.

↖ or process
A **Markov chain** is a directed graph (pot. with loops) with edges weighted by T s.t. $\forall x$

$$\sum_{x'} T_{x \rightarrow x'} = 1.$$

We have $P^{(t+1)}(X^{(t+1)} = x') = \sum_x P^{(t)}(X^{(t)} = x) T_{x \rightarrow x'}$.

A distribution π is **stationary** if $\pi^{(t+1)} = \pi^{(t)}$.

A Markov chain is **regular** if there is a k s.t. for every x, x' , the probability of getting from x to x' in exactly k steps is positive.

Theorem: A regular Markov chain converges to a unique stationary distribution, regardless of the initial distribution.

Note that if any two states are connected by paths with $T > 0$, including a state to itself, then the Markov chain is regular.

↖ Markov chain Monte Carlo MCMC

Algorithm (Given a probability distribution P , compute $P(x \in S)$):

- 1) Construct a Markov chain with limiting distribution P ,
- 2) Sample $x^{(0)}$ from any $P^{(0)}$,
- 3) Iteratively generate $x^{(t+1)}$ from T .

When $P^{(t)} \approx P$ we say that the chain has **mixed**.

In general, it is impossible to prove that a chain has mixed, but we can often show that it has not mixed by comparing different runs of the algorithm. Note that once the Markov chain has mixed, all further samples are sampled from the limiting distributions. But also note that nearby samples are correlated!

↑ the faster the convergence, the less correlated

Now consider the Gibbs distribution $P_{\Phi}(X_1, \dots, X_n)$. We now build a Markov chain called **Gibbs chain** to sample from P_{Φ} :

The state space is the set of assignments to $\{X_1, \dots, X_n\}$, and we the transition function from x to x' is defined by $x_i \sim P(X_i | x_{-i})$.

↑ all sampled assignments except x_i , (the current sampled assignments for $j < i$ and the previous ones for $j > i$)

Note that
$$P_{\Phi}(X_i | x_{-i}) = \frac{P_{\Phi}(X_i, x_{-i})}{P_{\Phi}(x_{-i})} = \frac{\tilde{P}_{\Phi}(X_i, x_{-i})}{\tilde{P}_{\Phi}(x_{-i})}$$

complete assignment, i.e. the product of factors

note that $X_i \setminus X_i$ cancel out

The Gibbs chain is not in general regular, e.g. for $Y = \text{XOR}(X_1, X_2)$. But if all factors are positive then the Gibbs chain is regular. Mixing may still be slow!

The **Metropolis-Hastings algorithm** gives a method for finding regular Markov chains with a desired limiting distribution.

We say that a chain is **reversible** for π if $\pi(x) T_{x \rightarrow x'} = \pi(x') T_{x' \rightarrow x}$. We call the the **detailed balance equation**.

Proposition: If T is regular and reversible for π then its limiting distribution is π .

Now consider a proposal distribution $Q_{x \rightarrow x'}$ and an acceptance probability $A_{x \rightarrow x'}$.

We define

$$T_{x \rightarrow x'} = \begin{cases} Q_{x \rightarrow x'} A_{x \rightarrow x'} & \text{if } x \neq x' \\ Q_{x \rightarrow x} + \sum_{x' \neq x} Q_{x \rightarrow x'} (1 - A_{x \rightarrow x'}) & \text{o/w} \end{cases}$$

we propose $x \rightarrow x'$

proposal is accepted

Note that the detailed balance equation always holds for $x \neq x'$. We want to find A s.t. the DBE holds:

$$\pi(x) Q_{x \rightarrow x'} A_{x \rightarrow x'} = \pi(x') Q_{x' \rightarrow x} A_{x' \rightarrow x}$$

$$\Leftrightarrow \frac{A_{x \rightarrow x'}}{A_{x' \rightarrow x}} = \frac{\pi(x')}{\pi(x)} \frac{Q_{x' \rightarrow x}}{Q_{x \rightarrow x'}} = q$$

Assume wlog that $q < 1$ and set $A_{x \rightarrow x'} = q$, $A_{x' \rightarrow x} = 1$. Hence,

$$A_{x \rightarrow x'} = \min\left(1, \frac{\pi(x')}{\pi(x)} \frac{Q_{x' \rightarrow x}}{Q_{x \rightarrow x'}}\right).$$

Note that Q must satisfy $Q_{x \rightarrow x'} > 0 \Rightarrow Q_{x' \rightarrow x} > 0$. Q should try to spread out to improve mixing, but then $A_{x \rightarrow x'}$ may become low. In general, finding a good $Q_{x \rightarrow x'}$ is an art!

Summary

Marginals

less fragile

confidence in answers

supports decision making

vs.

MAP

coherent joint assignment

more tractable model classes

same theoretical guarantees

approx. inferences

errors are often attenuated

ability to gauge whether algorithm is working

algorithms

Exact inference

Loopy message passing

Sampling methods

Exact inference

Optimisation

Search-based (e.g. sampling)

approx. inference problems

highly connected

strong influences (coupling)

opposing influences

multiple peaks in likelihood

→ identify problem regions,
use more expensive algorithm there